Data Integration

Whenever the data we need for our analytic goals are from different sources, before we can perform the data analytics, we need to integrate the data sources into one dataset that we need for our analytic goals. There are many challenges that you need to overcome before integration is possible. These challenges could be due to organizational privacy and security challenges that restrict our data accessibility. But even assuming that these challenges are not in the way when different data sources need to be integrated, they arise because each data source is collected and structured based on the needs, standards, technology, and opinions of the people who have collected them. Regardless of correctness, there are always differences in the ways that the data is structured and because of that, data integration becomes challenging.

Data integration Approaches

Data integration may happen in two different directions.

- The first is by **adding attributes**; we might want to supplement a dataset with more describing attributes. In this direction, we have all the data objects that we need, but other sources might be able to enrich our dataset.
- The second is by **adding data objects**; we might have multiple sources of data with distinct data objects, and integrating them will lead to a population with more data objects that represent the population we want to analyze.

Challenges in Data Integration

Challenge 1 – entity identification

- The entity identification challenge may occur when the data sources are being integrated by adding attributes.
- The challenge is that the data objects in all the data sources are the same real-world entities with the same definitions of data objects, but they are not easy to connect due to the unique identifiers in the data sources.
- For instance, in the data integration example section, the sales department and the marketing department did not use a central customer unique identifier for all their customers.
- Due to this lack of data management, when they want to integrate the data, they will have to figure out which customer is which in the data sources.

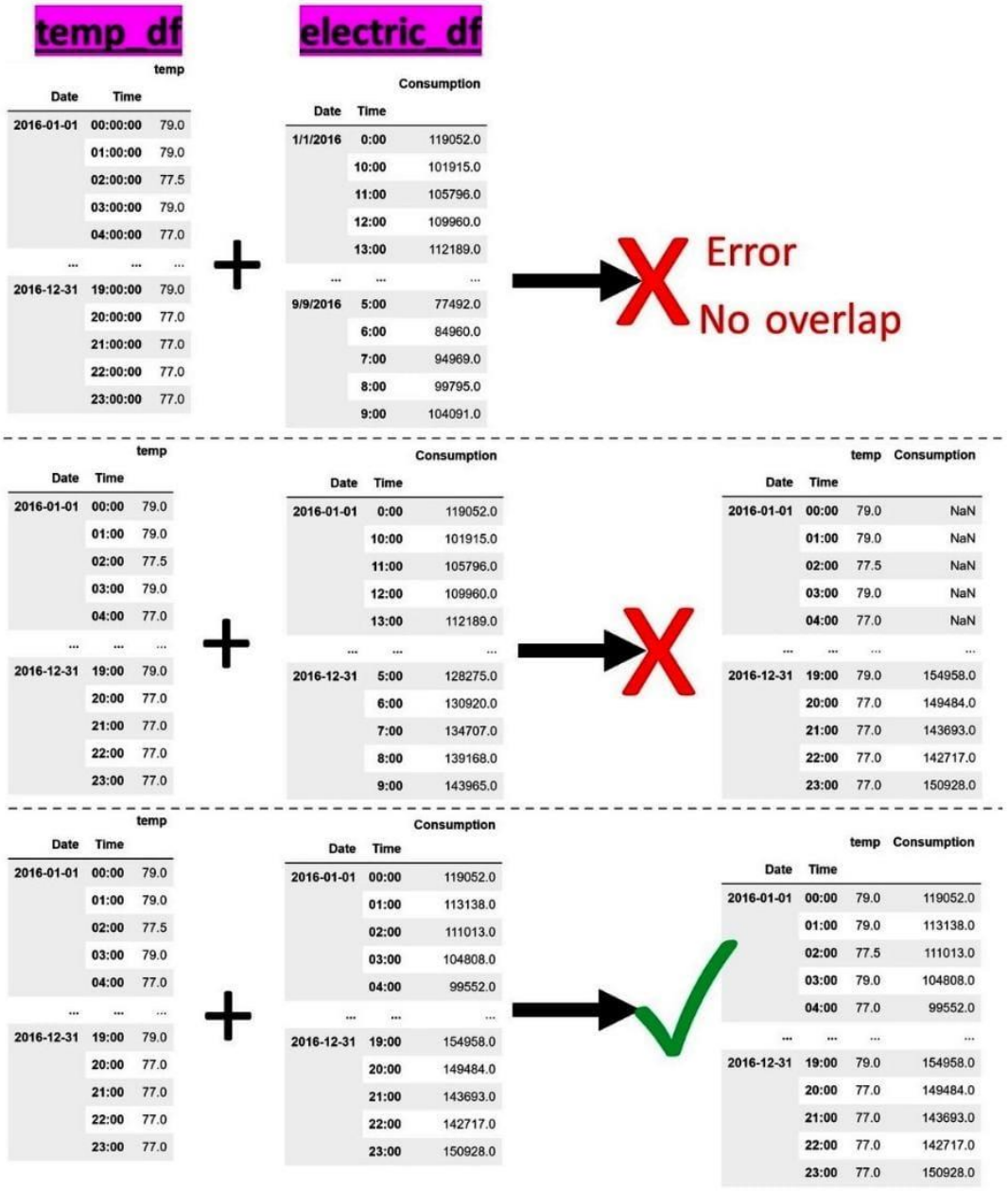
Challenge 2 – unwise data collection

- This data integration challenge happens, as its name suggests, due to unwise data collection.
- For instance, instead of using a centralized database, the data of different data objects is stored in multiple files.

- Regardless, in these situations, our goal is to make sure that the data is integrated into one standard data structure. This type of data integration challenge happens when data objects are being added.

Challenge 3 – index mismatched formatting

- When we start integrating data sources by adding attributes, we will use the pandas DataFrame .join() function to connect the rows of two DataFrames that have the same indices.
- To use this valuable function, the integrating DataFrames needs to have the same index formatting; otherwise, the function will not connect the rows.
- For example, the following figure shows three attempts of combining two DataFrames: temp_df and electric_df.



- temp_df contains the hourly temperature (temp) of 2016, while electric_df carries the hourly electricity consumption (consumption) for the same year.
- The first two attempts (the top one and the middle one) are unsuccessful due to the index mismatched formatting challenge.
- For instance, consider the attempt at the top; while both DataFrames are indexed with Date and Time and both show the same Date and Time, attempting the .join() function will produce a "cannot join with no overlapping index names" error.
- The attempt to integrate was unsuccessful because the index formatting from the two DataFrames is not the same:

In the preceding diagram, while the attempt in the middle is better than the one at the top, it is still unsuccessful. Pay close attention and see if you can figure out why there are so many NaNs in the output of the integration.

Challenge 4 – aggregation mismatch

- This challenge occurs when integrating data sources by adding attributes.
- When integrating time series data sources whose time intervals are not identical, this challenge arises.
- For example, if the two DataFrames presented in the following figure are to be integrated, not only do we have to address the challenge of index mismatch formatting, but we will also need to face the aggregation mismatch challenge.
- This is because temp_df carries the hourly temperature data but electric_df carries the electricity consumption of every half an hour.
- To deal with this challenge, we will have to restructure one source or both sources to get them to have the same level of data aggregation.

temp_df

	Timestamp	temp
0	2016-01-01T00:00:00	79.0
1	2016-01-01T01:00:00	79.0
2	2016-01-01T02:00:00	77.5
3	2016-01-01T03:00:00	79.0
4	2016-01-01T04:00:00	77.0
...	...	...
8726	2016-12-31T19:00:00	79.0
8727	2016-12-31T20:00:00	77.0
8728	2016-12-31T21:00:00	77.0
8729	2016-12-31T22:00:00	77.0
8730	2016-12-31T23:00:00	77.0

electric_df

	Date	Time	Consumption
0	12/1/2017	0:00:00	72650.0
1	12/1/2017	0:30:00	70553.0
2	12/1/2017	1:00:00	68277.0
3	12/1/2017	1:30:00	67611.0
4	12/1/2017	2:00:00	67388.0
...	...	...	...
19051	1/1/2016	21:30	56059.0
19052	1/1/2016	22:00	55107.0
19053	1/1/2016	22:30	55609.0
19054	1/1/2016	23:00	58199.0
19055	1/1/2016	23:30	57539.0

Challenge 5 – duplicate data objects

- This challenge occurs when we're integrating data sources by adding data objects.
- When the sources contain data objects that are also in the other sources, when the data sources are integrated, there will be duplicates of the same data objects in the integrated dataset.
- For example, imagine a hospital that provides different kinds of healthcare services. For a project, we need to gather the socioeconomic data of all of the patients in the hospital.
- The imaginary hospital does not have a centralized database, so all of the departments are tasked with returning a dataset containing all the patients they have provided services for.
- After integrating all of the datasets from different departments, you should expect that there are multiple rows for the patients that had to receive care from different departments in the hospital.

Challenge 6 – data redundancy

- This challenge's name seems to be appropriate for the previous challenge as well, but in the literature, the term data redundancy is used for a unique situation.
- Unlike the previous challenge, this challenge may be faced when you're integrating data sources by adding attributes.
- As the name suggests, after data integration, some of the attributes may be redundant.
- This redundancy could be shallow as there are two attributes with different titles but the same data.
- Or, it could be deeper. In deeper data redundancy cases, the redundant attribute does not have the same title, nor is its data the same as one of the other attributes, but the values of the redundant attribute can be derived from the other attributes.
- For example, after integrating data sources into a dataset of customers, we have the following seven attributes: age, average order $, days from the last visit, weekly visit frequency, weekly $ purchase, and satisfaction score. If we use all seven attributes to cluster customers, we have made a mistake regarding data redundancy. Here, the weekly visit frequency, weekly $ purchase, and average order $ attributes are distinct but the value of weekly $ purchase can be derived from weekly visit frequency and average order $. By doing so, inadvertently, we will have given the information regarding the customer's visit and their purchase amount more weight in the clustering analysis.

Data Redundancy

Another important step of data preprocessing that is not concerned with data cleaning; this is known as data reduction. To successfully perform analytics, we need to be able to recognize situations where data reduction is necessary and know the best techniques and the how-to of their implementation.

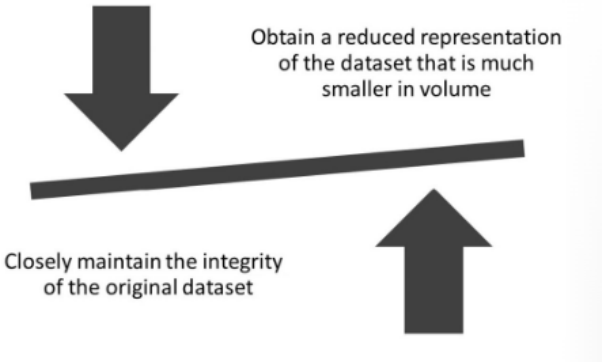
The distinction between data reduction and data redundancy

- While data redundancy and data reduction have very similar names and their terms use words that have connected meanings, the concepts are very different.
- Data redundancy is about having the same information presented under more than one attribute. As we saw, this can happen when we integrate data sources.
- However, data reduction is about reducing the size of data due to one of the following three reasons:
    - **High-Dimensional Visualizations**: When we have to pack more than three to five dimensions into one visual, we will reach the human limitation of comprehension.
    - **Computational Cost:** Datasets that are too large may require too much computation. This might be the case for algorithmic approaches.
    - **Curse of Dimensionality:** Some of the statistical approaches become incapable of finding meaningful patterns in the data because there are too many attributes
- In other words, data redundancy is a characteristic that a dataset may have. This characteristic is about having redundant data in the dataset, so we may have to take some actions.
- On the other hand, data reduction is a set of actions that we can take to reduce the size of data due to the aforementioned reasons.

The objectives of data reduction

Successful data reduction seeks to achieve the following two objectives at the same time.

- First, data reduction seeks to obtain a reduced representation of the dataset that is much smaller in volume.
- Second, it tries to closely maintain the integrity of the original data, which means making sure that data reduction will not lead to including bias and critical information being lost in the data.



- As shown in the following diagram, these two objectives can be contradictory and when performing data reduction actions, the two objectives must be taken into consideration at the same time so that one is not overshadowed by the other:

Types of data reduction

There are two types of data reduction methods.

- **Numerosity data reduction:** performs data reduction by reducing the number of data objects or rows in a dataset
- **Dimensionality data reduction:** performs data reduction by reducing the number of dimensions or attributes in a dataset

Numerosity reduction methods

- **Random Sampling:** Randomly selecting some of the data objects to avoid unaffordable computational costs.

- **Stratified Sampling:** Randomly selecting some of the data objects to avoid the unaffordable computational costs, all the while maintaining the ratio representation of the sub-populations in the sample.

- **Random Over/Under Sampling:** Randomly selecting some of the data objects to avoid the unaffordable computational costs, all the while creating a prescribed representation of the sub-populations in the sample.

Dimensionality reduction methods

- **Linear Regression:** Using regression analysis to investigate the predictive power of independent attributes to predict a specific dependent attribute
- **Decision Tree:** Using the decision tree algorithm to investigate the predictive power of the independent attributes to predict a specific dependent attribute
- **Random Forest:** Using the random forest algorithm to investigate the predictive power of the independent attributes to predict a specific dependent attribute
- **Brute-force Computational Dimension Reduction:** Computational experimentations to figure out the best subset of independent attributes that leads to the most successful prediction of the dependent attribute
- **Principal Component Analysis (PCA):** Representing the data by transforming the axes in such ways that most of the variation in the data is explained by the first attributes and the attributes are orthogonal to one another
- **Functional Data Analysis (FDA):** Representing the data using fewer points using functional representation